

AI Data Science Workbench

A Governed, Task-Agnostic Machine Learning Pipeline

Project report

Machine Learning · Amrita School of Business (MBA), Term IV 2026

Table of contents

1. Executive summary
2. The problem this project addresses
3. Scope and objectives
4. System architecture
5. Data understanding and preparation
6. Feature engineering
7. The model catalogue
8. Training and hyperparameter tuning
9. Evaluation and champion selection
10. Results
11. Explainability
12. The governance layer
13. The trust layer
14. Testing strategy
15. Application layers
16. Engineering lessons
17. Limitations
18. Future work
19. Appendix A — Command reference
20. Appendix B — Artifact inventory
21. Appendix C — Repository map

1. Executive summary

This project is a **machine learning pipeline that treats governance as a first-class requirement rather than documentation added afterwards**. It runs two complete case studies — California house-price regression and Telco customer-churn classification — through one identical code

path, and produces, for every run, a complete record of which data, which parameters, and whose approval produced each number.

The central claim is deliberately *not* "the model is accurate". It is:

The result is reproducible, the model choice is documented, leakage is structurally prevented rather than merely avoided, and the model's own bias is measured and published alongside its accuracy.

Headline results. On California housing, the champion is **Ridge regression (alpha = 100)** with a test RMSE of **\$66,681** and **R² 0.667**. On Telco churn, the champion is **Logistic Regression (C = 2.154)** with a test **ROC-AUC of 0.8448** and accuracy of **0.811**. In both cases the interpretable model won — on churn it matched the Random Forest's discrimination (0.8446 AUC) while being substantially more precise, and on housing it beat the unregularised baseline by a small but real margin.

Scale of the system. Twelve governed pipeline stages; six models in an extensible catalogue; 136 automated tests; five standing methodological decisions recorded in the repository before any model was trained; ten audit event types; a four-part trust audit producing a model card that refuses to state claims it cannot evidence.

What distinguishes this from a notebook. A notebook records *what ran*. It does not record *why that model was chosen over the alternatives*, cannot prove the test set was untouched during selection, and will happily return an excellent score from a scaler fitted on all rows before splitting. Each of those failure modes is closed here by construction, not by discipline — the training function physically cannot fit an unapproved model, and the code raises an exception if the target appears among the predictors.

2. The problem this project addresses

2.1 The reproducibility gap

A conventional analysis notebook has four structural weaknesses that only surface when someone else — a reviewer, an auditor, or the author six months later — tries to trust the result.

It records what ran, never why. A cell that fits three models and prints their scores does not capture the reasoning that selected one. That reasoning lives in the analyst's head, and it evaporates.

Results drift silently. Without a fixed random seed, a declared split policy, and a hash of the input file, two runs of "the same" analysis can differ, and there is no way to tell whether the difference came from the data, the code, or chance.

Leakage produces better scores, not errors. A `StandardScaler` fitted on the full dataset before splitting will never raise an exception. It will quietly leak the test set's distribution into training and return a flattering, wrong number. This is the single most common failure in applied machine learning precisely because it is invisible.

Nothing gates the pipeline. There is no point at which an unreviewed model is stopped from reaching the results section.

2.2 The response

Each weakness above is met with a structural mechanism rather than a convention:

Weakness	Mechanism
No record of <i>why</i>	Five standing decisions recorded before training; a model proposal document; an approval artifact naming what was accepted
Silent drift	<code>RANDOM_STATE = 42</code> project-wide; declared 70/15/15 split; SHA-256 hash of the training file stored on every registry entry
Invisible leakage	Every fitted statistic learned from the train split only; the fitted preprocessor travels inside the model bundle; <code>split_x_y()</code> raises if the target is among the predictors
No gate	<code>require_model_approved()</code> raises <code>GovernanceError</code> before any estimator is fit

3. Scope and objectives

3.1 Objectives

22. Build a pipeline whose every stage declares its inputs and outputs, and which refuses to run a stage whose prerequisites are missing.
23. Demonstrate that the design is genuinely general by running two different learning tasks — regression and binary classification — through the same code, with no task-specific branches in the orchestration layer.
24. Prevent data leakage structurally, so that a leak is a crash rather than a good score.
25. Produce, for every run, a governance trail sufficient to reconstruct the analysis.
26. Audit the champion model for subgroup fairness, genuine feature dependence, and

probability calibration — and publish the findings in a model card that states what it does not know.

3.2 The two case studies

	California Housing	Telco Customer Churn
Task	Regression	Binary classification
Target	<code>median_house_value</code>	<code>Churn</code>
Positive class	not applicable	"Yes"
Selection metric	<code>rmse</code> (lower is better)	<code>roc_auc</code> (higher is better)
Stratified on	income band	target class
Rows (train/val/test)	~14,400 / ~3,100 / ~3,100	4,930 / 1,056 / 1,057
Activated by	default	<code>WORKBENCH_PROJECT=churn</code>

Running both is the strongest available test of the architecture. A pipeline that only ever handles regression can hide task-specific assumptions in shared code; forcing a classification problem through the same path exposes them immediately.

3.3 A third mode: bring your own dataset

Beyond the two curated projects, the workbench accepts an arbitrary uploaded CSV. An AutoML detection layer profiles every column, guesses the target, infers whether the task is regression or classification, identifies categorical predictors, and flags identifier columns to drop. These are explicitly **heuristics with a human in the loop** — the interface presents each guess as an editable suggestion, and nothing trains until the user confirms. This matters because the guess is genuinely fallible: on the raw housing file, the target heuristic falls back to the last column and selects `ocean_proximity` instead of `median_house_value`.

4. System architecture

4.1 The twelve stages

The pipeline is a registry of twelve stages. The registry order *is* the canonical pipeline order.

#	Stage	What it does	Key output
1	<code>ingest</code>	Load raw data, profile every column, hash the file	<code>data/raw/housing.csv</code>
2	<code>preprocess</code>	Clean, then split 70/15/15, then fit transformers	<code>data/splits/{train,validation,test}.csv</code>

3	<code>understand</code>	Schema conformance, missingness, logical consistency	<code>results/data_understanding/</code>
4	<code>eda</code>	Distributions, correlations, outliers, non-linearity screens	<code>results/eda/eda_report.txt</code>
5	<code>features</code>	Propose features, then execute the automatic ones	<code>data/engineered/*.csv</code>
6	<code>propose</code>	Build the model proposal document	<code>governance/approvals/model_proposal.json</code>
7	<code>approve</code>	Governance gate — record what is approved	<code>governance/approvals/model_approval.json</code>
8	<code>train</code>	Fit and tune every approved model	<code>models/model_registry.json</code>
9	<code>evaluate</code>	Compare on validation, promote champion, score test once	<code>artifacts/final_model_selection.json</code>
10	<code>explain</code>	Coefficients, drivers, executive briefing	<code>artifacts/explainability/</code>
11	<code>trust</code>	Verification gate — fairness, importance, calibration, card	<code>artifacts/trust/model_card.md</code>
12	<code>report</code>	Self-contained HTML report	<code>artifacts/reports/project_report.html</code>

Two stages are gates. Stage 7 (`approve`) is where a human decides what may be trained. Stage 11 (`trust`) is where the promoted model is interrogated before it is believed.

4.2 The orchestrator

The orchestrator has three responsibilities, and deliberately no others — it changes no stage's behaviour, it only sequences them.

It refuses a stage whose declared inputs are missing, and names the prerequisite stage that produces them. This turns a confusing downstream `KeyError` into a clear statement about pipeline order.

It persists per-stage status to `artifacts/workflow_status.json` — last run timestamp, duration, status (`ok` / `failed` / `skipped`), and the outputs produced.

It supports resume and force. By default, a stage whose declared outputs already exist is skipped. `--force` re-runs regardless. Importantly, the *dependency check runs even for skipped stages*, so a resumed pipeline cannot silently proceed on a broken prerequisite.

One design decision here is worth stating because it looks like an inconsistency. The `report` stage depends on `explain`, not on `trust` — even though `trust` runs before it. The reason: the report predates the trust stage, and it must not start failing when a trust artifact happens to be absent. Because `trust` sits earlier in registry order, a full run still picks it up. The dependency graph encodes what is *required*, not merely what is *usual*.

4.3 Multi-project workspaces

A single environment variable selects the active project:

```
WORKBENCH_PROJECT=churn python main.py all
```

Internally, `config/paths.py` resolves a workspace root from that variable. Every data, model, artifact and governance directory hangs off that root, so two projects can never overwrite one another's outputs.

The California Housing showcase is a deliberate exception: it keeps a flat layout at the repository root, because its model registry stores absolute model paths and its graded report already lives there. Every other project — including the churn study and every uploaded dataset — is isolated under `workspaces/<slug>/`.

A hazard worth documenting. `config/paths.py` reads the environment variable *once, at module import*. Any code that registers a new project and then runs it must set the variable **before** importing the pipeline modules. The web bridge handles this by computing the project slug with a small local duplicate of the slug function, setting the environment variable, and only then importing the pipeline. Getting this order wrong produces a pipeline that writes into the previously active project's workspace — a silent, destructive failure.

4.4 Three entry points, one engine

Layer	Purpose	Relationship to the pipeline
CLI (<code>main.py</code>)	Primary interface; 19 subcommands	Calls the pipelines directly
Streamlit app	Nine-page analyst GUI, including the retention simulator	Reads artifacts; triggers stages
Next.js console	Ten-stage review journey over uploaded datasets	Read-only over artifacts; spawns Python as a child process

All three read the same artifacts. None of them re-implements analysis. This was not free — an earlier version had the CLI and the web path each maintaining their own hand-written list of pipeline calls, and those lists drifted: the web path generated the driver table and the HTML report, the CLI command did not, so the same dataset produced different deliverables depending on how it was launched. Both now call one shared composed pipeline.

5. Data understanding and preparation

5.1 The semantic schema

The housing project is governed by an explicit schema that declares, per column, its role, semantic type, expected storage type, nullability, permitted minimum and maximum, whether it must be a whole number, and which preprocessing group it belongs to.

This schema is not decorative. The EDA stage *refuses to run* on a dataset the schema does not describe. Unguarded, such a dataset surfaced as `KeyError: 'ocean_proximity'` from three frames deep, which reads as a broken pipeline. It is not: it is a dataset the governed schema does not cover, and the guard says so.

A subtle detail in that guard: it runs **before** the stage creates its output directories. A refused stage must leave nothing behind, because an empty `results/eda/` directory is indistinguishable from a completed one to anything that probes the filesystem — including the web application's "has this stage run?" check.

5.2 Validation

The validation stage checks the raw data against the schema and produces an explicit status: `FAILED`, `PASSED WITH WARNINGS`, or `PASSED`, alongside per-issue and per-observation CSVs. Validation findings are recorded, not silently repaired.

5.3 Exploratory data analysis

EDA runs on the **training split only**. This is not a stylistic preference — looking at the test set to decide which features to engineer is leakage through the analyst.

The stage produces summary statistics, distribution shapes, missingness reports, categorical summaries, a correlation matrix, target correlations, domain-anomaly detection, and outlier reports by two methods (IQR with a 1.5 multiplier, and modified z-score with a 3.5 threshold). A second analysis-summary stage screens predictors for marginal association, non-linearity (via a curvature score), and multicollinearity above a 0.80 pairwise correlation threshold, then writes a plain-language conclusion.

Two deliberate deviations from the reference implementation are recorded in the code: LOWESS smoothing is replaced by a dependency-free binned-mean smoother, to avoid adding

`statsmodels` as a dependency for one chart; and a several-hundred-line HTML/CSS presentation layer is replaced by CSV plus Markdown outputs, which are diffable and machine-readable.

All figures are written with `savefig`, never `show` — the pipeline must run non-interactively.

5.4 The splitting policy

Parameter	Value
Train fraction	0.70
Validation fraction	0.15
Test fraction	0.15
Random state	42, fixed project-wide
Housing stratification	Income band, bins <code>[0, 1.5, 3.0, 4.5, 6.0, inf)</code>
Churn stratification	Target class

Three splits rather than two is the crux of the methodology. The validation split exists so that hyperparameter tuning and model comparison have somewhere to happen that is neither the training data nor the test data.

5.5 Leakage prevention

The governing invariant, stated verbatim in the preprocessing module:

The split happens BEFORE any fitted transformation. Imputer, encoder, and scaler are fit on the training split only and reused unchanged on validation and test.

The modelling design matrix is built by a `ModelingPreprocessor` with three guarantees:

Every fitted statistic comes from train only. Medians for imputation, means and standard deviations for standardisation, and the category levels for one-hot encoding are all learned from the training split.

The fitted preprocessor is pickled inside the model bundle. Each saved model is a dictionary containing the estimator, the preprocessor, the predictor columns, the feature names, the task, and the positive class. Evaluation, explainability and the trust audit therefore transform data *identically* to training — not similarly, but with the same object. It also means the stored per-feature standard deviations are available later to convert standardised coefficients back into raw dollar effects.

The target cannot enter the predictors. `split_x_y()` raises a `ValueError` if the target column appears in the predictor list. Leakage becomes a crash.

The transformation order is:

Column type	Treatment
-------------	-----------

Numeric	Median-impute using the train median
Numeric	Standardise using the train mean and standard deviation
Categorical	One-hot encode using train categories only
Unseen category level	Encoded as an all-zero row — never an error

Two guards deserve mention. A column whose training standard deviation is zero receives a divisor of 1.0 rather than dividing by zero, which would otherwise produce a silent column of NaN or infinity. And a configured categorical column is treated as categorical even when it stores numeric codes — churn's `SeniorCitizen` is coded 0/1 but is a category, not a quantity.

6. Feature engineering

Feature engineering is split into two stages that are deliberately separate: a **proposal** stage that creates no features at all, and an **execution** stage that builds only the approved ones.

The proposal organises candidates into five groups — deterministic, domain, statistical, machine learning, and inferential — and marks each as automatic or requiring review. Only ratios and log transforms are marked automatic, because they are pure row-wise functions: they need no fitting, so they cannot leak information across splits.

Six ratio features are defined (rooms per household, bedrooms per room, population per household, bedrooms per household, rooms per person, bedrooms per person) along with log transforms. Two correctness details are enforced by tests: division uses a safe helper that yields NaN rather than infinity on a zero denominator, and log features use `log1p`, so a zero value maps to zero rather than negative infinity.

7. The model catalogue

Models are registered as `ModelSpec` dataclasses in a single catalogue. Each entry records the scikit-learn class path as a **string**, its search space, its default parameters, and its governance-facing prose — strengths, limitations, interpretability notes, and a suitability rationale.

Model	Family	Task	Tuned parameter	Fixed settings
LinearRegression	linear	Regression	none	transparent baseline
Ridge	linear	Regression	<code>alpha</code> , log 1e-3 to 1e3	L2 shrinkage
Lasso	linear	Regression	<code>alpha</code> , log 1e-4 to 1e1	<code>max_iter=10000</code> , <code>tol=0.1</code>

LogisticRegression	linear	Classification	C, log 1e-2 to 1e2	liblinear, max_iter=2000
DecisionTreeClassifier	tree	Classification	max_depth, 2 to 12	class_weight='balanced'
RandomForestClassifier	ensemble	Classification	max_depth, 4 to 16	300 trees, balanced, n_jobs=-1

Grids contain 25 points, log-spaced or integer as appropriate.

Every model has exactly one tunable hyperparameter. This is a design constraint, not a coincidence. It allows one validation sweep to serve both tasks, so there is no separate regression tuner and classification tuner that can drift apart.

Why the linear family leads. This is a recorded governance decision. Linear coefficients read directly as signed effects a manager can act on. The tree and forest were added for the churn study specifically as the accuracy benchmark the interpretable model has to match — the argument for interpretability is only honest if the accuracy cost is measured.

Extensibility. Because estimators are resolved from a string path at runtime, adding a model family edits the catalogue only. The trainer, evaluator, and reporter import no estimator directly.

Two implementation details from the catalogue are worth recording. Lasso's `tol` is set to 0.1 — this looks alarmingly loose until one notes it is relative to a dollar-scale target whose median is around \$100,000, so the tolerance is far below any meaningful precision, and it converges in seconds rather than minutes with an identical solution. And Logistic Regression is pinned to the `liblinear` solver, which cannot fit multiclass problems; a targeted override switches to `lbfgs` only when the target has three or more classes, leaving every binary result bit-identical.

8. Training and hyperparameter tuning

8.1 The governance guard

The first statement in the training function is the approval check. An estimator absent from `model_approval.json` raises `GovernanceError` before any fitting occurs. Separately, the design matrix is built only from approved predictor columns, so the target and any non-approved column are excluded *by construction* rather than by remembering to exclude them.

8.2 Hold-out tuning, not k-fold

The hyperparameter is swept on the dedicated validation split — **not** k-fold cross-validation over train plus validation. This is a recorded standing decision (`tune_on_validation_not_kfold`).

The rationale: one split, one purpose. Reusing training rows to select a hyperparameter feeds the selection back into the data that produced it. The trade-off is honestly acknowledged — a single hold-out split gives a higher-variance estimate than k-fold, and with a smaller dataset k-fold would be the better choice. Here the splits are large enough that the cleaner separation is worth more than the variance reduction.

8.3 One loop, two directions

A single dictionary, `METRIC_HIGHER_IS_BETTER`, supplies the direction of improvement for every metric. RMSE (lower wins) and ROC-AUC (higher wins) therefore share one tuning loop and one champion-selection routine.

The comparison helper handles NaN explicitly, and the reason is instructive. Several metrics here return NaN *by design* when they are undefined: ROC-AUC when a class is absent from a split, MAPE when an actual value is zero. In Python, every comparison against NaN is `False`. A naive `>` or `<` therefore produces two wrong behaviours: a NaN incumbent can never be displaced by a real score, and the search silently keeps whichever candidate happened to be evaluated first.

The rule implemented is that an undefined score is treated as no score at all: it never wins, and it always loses to a real one. This bug was **ordering-dependent** — a NaN in the middle of the candidate list selected correctly, a NaN first did not — which is precisely why it survived earlier testing, and why the test suite now covers NaN in first, middle, and last position separately.

8.4 Persistence

Every trained model produces a versioned directory containing the joblib bundle and a metadata file recording parameters, the full tuning history, train and validation metrics, training duration, row and feature counts, the predictor columns, the data hash, and the random seed. Versions auto-increment (`v001`, `v002`); nothing is overwritten, so earlier results remain auditable.

9. Evaluation and champion selection

The promotion sequence is:

27. Sweep 25 candidate hyperparameter values, scoring each on **validation**.
28. Refit the best parameters on **train**.
29. Compare all trained models on **validation**.
30. Promote the best as champion and record it in the registry.
31. Score the champion **once** on the test split.

The test split is loaded exactly once, and only the champion's predictions are scored against it. This preserves the untouched-test doctrine: the test set measures generalisation, and a set

consulted during selection no longer measures generalisation — it becomes a second validation set with an inflated reputation.

The evaluator also writes a comparison table with an explicit rank column and a plain-language selection narrative explaining, in a sentence a non-specialist can read, why the champion won.

9.1 On the trust layer reading test again

The trust stage reads the test split a second time. This is deliberate and is not a breach of the doctrine. The champion is already frozen before the trust stage runs, and nothing in the trust layer feeds back into model selection or tuning. Reading test again produces a *report on a decision already made*, not the decision.

10. Results

10.1 Side by side

	California Housing	Telco Customer Churn
Task	Regression	Binary classification
Candidates trained	6 (3 models, 2 versions each)	6 (3 models, 2 versions each)
Selection metric	<code>rmse</code>	<code>roc_auc</code>
Champion	Ridge v001, alpha = 100	LogisticRegression v001, C = 2.154
Validation	RMSE \$64,787 · R ² 0.680	ROC-AUC 0.8455 · Accuracy 0.808
Test (opened once)	RMSE \$66,681 · R ² 0.667	ROC-AUC 0.8448 · Accuracy 0.811
Runner-up	Lasso / LinearRegression, RMSE \$64,887	RandomForest, ROC-AUC 0.8446

10.2 California housing

The champion is Ridge with `alpha = 100`, selected on validation RMSE. Test performance is RMSE \$66,681, MAE \$48,731, R² 0.667, MAPE 28.3%.

The margin is small and is reported as such. Ridge beat plain LinearRegression by approximately \$100 of validation RMSE — \$64,787 against \$64,887. Lasso landed at the same \$64,887. Three models therefore sit within \$100 of each other. L2 shrinkage helped slightly with the multicollinearity among the room, bedroom, population and household counts; it did not transform the problem. Overstating this would be dishonest, and the tie is arguably the more interesting finding: it says the signal in these features is close to fully extracted by a linear model, and that further gains require a different model class or better features, not better regularisation.

The validation-to-test movement — \$64,787 to \$66,681 — is a modest and expected degradation, consistent with a model that has not been over-selected on validation.

Top drivers, from standardised coefficients: `median_income` (positive, by a wide margin), `latitude` (negative), `longitude` (negative), `log_population` (negative), and `ocean_proximity_INLAND` (negative). The geographic terms are the model doing its best to approximate a coastal price gradient with a plane, which is exactly the non-linearity the EDA stage flagged and the linear family cannot capture.

10.3 Telco customer churn

The champion is Logistic Regression with `C = 2.154`, `solver = liblinear`, `max_iter = 2000`, trained on 4,930 rows and evaluated on 1,057.

Model	Val ROC-AUC	Val precision	Val recall	Val F1
LogisticRegression	0.8455	0.6498	0.5964	0.6220
RandomForestClassifier	0.8446	0.5011	0.8357	0.6265
DecisionTreeClassifier	0.8386	0.5191	0.7750	0.6218

Interpretability cost nothing here. The Random Forest matched the discrimination (0.8446 versus 0.8455 — a difference well inside noise) but at a very different operating point: it caught far more churners (recall 0.836 against 0.596) at the cost of being wrong about half the time it fired (precision 0.501 against 0.650). Which is preferable depends entirely on the cost of a wasted retention offer versus a lost customer — and that is a business decision, not a modelling one. Because AUC is threshold-independent and the two models tie on it, the tie-break went to the model whose coefficients convert directly into odds ratios.

Test performance is ROC-AUC 0.8448, accuracy 0.811, precision 0.687, recall 0.530, F1 0.598.

The validation-to-test movement is 0.8455 to 0.8448 — essentially nil, which is the clearest available evidence that the selection procedure did not overfit the validation split.

10.4 The retention simulator

The churn work includes a simulator that converts predicted probabilities into a campaign business case. Its model is stated explicitly so it can be challenged:

```
targeted          = customers where P(churn) >= threshold
expected_churners = sum of P(churn) over the targeted
retained          = expected_churners * acceptance_rate
revenue_saved     = retained * monthly_revenue * horizon_months
campaign_cost     = sum over targeted of (monthly_revenue * discount *
horizon_months)
net_value         = revenue_saved - campaign_cost
roi              = net_value / campaign_cost
```

The analyst controls four levers: risk threshold, discount offered as a percentage of the monthly bill, assumed acceptance rate, and value horizon in months.

This is where calibration stops being an academic concern. The simulator *multiplies a predicted probability by a cost*. If the probabilities run twenty points high, the ROI is twenty points of fiction — and the AUC, which only measures ranking, would not reveal it.

The simulator carries its own runnable self-check asserting the accounting identities: the empty-target case yields zero ROI rather than a division by zero, retained never exceeds expected churners, `net_value` equals `revenue_saved` minus `campaign_cost`, and outcomes increase monotonically in the acceptance rate.

11. Explainability

Explainability is deliberately split into two modules with different remits.

The **housing-specific interpreter** produces the graded deliverables: a coefficient table that converts standardised coefficients back into raw dollar effects using the standard deviations stored in the preprocessor, a top-drivers chart, and an executive briefing in plain language.

The **dataset-agnostic driver module** handles uploaded projects. Its docstring records the reason it exists separately:

Running it against an uploaded dataset would emit fluent, confident analysis about the wrong domain.

It therefore keeps the parts that are true for any tabular model — the ranked coefficients or importances, the chart — and **generates no narrative at all**, because no narrative can be written without knowing what the target means. This is a recurring principle in the project: the system would rather produce less than produce something fluent and wrong.

SHAP is supported but optional, and is intentionally absent from `requirements.txt`. For a linear-family champion, standardised coefficients already provide an exact, additive, per-feature attribution; SHAP would add a heavy dependency to re-derive what is already exact.

12. The governance layer

12.1 Standing decisions

Five methodological decisions are recorded in the repository — with a decision statement, a rationale, and a date — before any model is trained:

Key	Decision
<code>linear_family_only</code>	Restrict the catalogue to the linear family
<code>split_early_test_untouched</code>	Split early; the test split stays untouched until one

	final evaluation
<code>median_imputation_after_split</code>	Impute after splitting, using train medians only
<code>tune_on_validation_not_kfold</code>	Tune on the validation split rather than k-fold
<code>governed_approvals_gate_training</code>	Training refuses any model not in the approval artifact

The point is that the methodology lives in the repository, not only in a report written afterwards. A reviewer can check whether the code honours the stated method.

12.2 The audit log

Every governed action appends a record — timestamp, actor, event type, payload — to `governance/audit/audit_log.jsonl`. Ten event types are emitted: feature approval, model approval, model proposal, training run, evaluation run, final model selection, explainability generation, trust audit, churn preparation, and auto preparation.

The log is append-only and flushed immediately, so a crash mid-pipeline still leaves a durable trail, and appending rather than rewriting keeps it tamper-evident.

12.3 Lineage

Each governed stage records a lineage node capturing the stage name, timestamp, the script that ran, every input file with its SHA-256 hash, every output file with its hash, a hash of the parameters, and the parameters themselves. Six nodes are recorded across a full run.

This is what allows a reviewer to trace any evaluated model back to the exact raw data and transformations behind it.

12.4 What is gated

Two things, both enforced in code rather than by convention:

Which estimators may be fit. Anything absent from `approved_models` raises `GovernanceError`.

Which columns enter the design matrix. The approved predictor set is the intersection of base schema predictors and approved engineered features that actually exist in the engineered training data. The target cannot appear, by construction.

12.5 The question this answers

Six months from now, can you prove which data, which parameters, and whose approval produced this number?

The registry names the champion and its version. The metadata file holds its parameters and full tuning history. The audit log records who approved it and when. The data hash identifies the exact input file. The lineage graph connects them.

13. The trust layer

The trust layer asks three questions the accuracy metric cannot answer, and adds a fourth derived measure.

13.1 Subgroup fairness

Every categorical column with at most ten distinct levels is audited. A column with more levels than that is not a subgroup — it is an identifier or a continuous measure. Groups smaller than thirty rows are reported but excluded from gap calculations, because a four-row group is noise, not a finding.

Metrics computed per group depend on the task: for binary classification, base rate, selection rate, true-positive rate, false-positive rate, precision and accuracy; for regression, signed mean error, MAE and RMSE.

R^2 is deliberately excluded from the regression group table, because it is measured against each group's *own* variance — a group whose targets are all similar scores terribly even when its errors are tiny.

Churn findings. Sixteen columns audited, none skipped, on 1,057 test rows. Ranked by the largest headline gap:

Column	Selection rate gap	TPR gap	Accuracy gap	Base rate gap
InternetService	0.4354	0.7158	0.1558	0.3145
Contract	—	0.6132	0.2637	0.4053

The [InternetService](#) audit also reports a **selection rate ratio of 0**, which means one group is *never predicted positive at all*.

13.2 Selection amplification

A raw fairness gap is ambiguous on its own. Groups genuinely churn at different rates, so a gap in selection rates may simply reflect reality. Selection amplification isolates the part the model added:

$$\text{selection_amplification} = \text{selection_rate_gap} - \text{base_rate_gap}$$

Positive means the model exaggerates a real difference; near zero means it tracks reality; negative means it under-separates groups that genuinely differ.

Column	Amplification	Reading
InternetService	+0.1208	The model flags one group about 12 percentage points more often

		than its real churn rate justifies
Contract	-0.0259	The model <i>narrows</i> a real gap

The code states this measure's limitation rather than hiding it: the two spreads are computed independently, so the widest-apart pair for selection rate need not be the widest-apart pair for base rate. It compares dispersion, not a matched pair, and is therefore a **triage signal for which column to examine first — not a bias test**.

13.3 Permutation importance

Coefficients and `feature_importances_` are the model's own account of itself. For tree ensembles that account is biased: impurity-based importance is computed from training-set impurity decrease, which systematically inflates high-cardinality and continuous features regardless of whether they help on unseen data.

Permutation importance instead shuffles each column on held-out data and measures the resulting drop in the selection metric — a direct measurement of dependence.

Churn findings: 46 features, ten shuffles each, scored against ROC-AUC. Only **18 are distinguishable from noise**.

Rank	Feature	Importance
1	<code>tenure</code>	0.1915 ± 0.0175
2	<code>MonthlyCharges</code>	0.0499
3	<code>TotalCharges</code>	0.0201

`tenure` is roughly four times the importance of the next feature. The noise threshold is intentionally strict — roughly three times the standard error of the mean — because a false "this matters" is more costly here than a false "cannot tell".

Two limitations are documented and unfixable at this design point: columns are permuted after one-hot encoding, so a categorical's total importance is not the sum of its dummies; and correlated columns share credit unpredictably, so the table must be read as groups rather than as a strict ranking.

When coefficients and permutation importance disagree, the disagreement is the finding — a feature with a large coefficient and no permutation importance is one the model cannot actually use on unseen data.

13.4 Calibration

Calibration asks: when the model says 0.8, does that happen 80% of the time?

Churn findings on 1,057 test rows across ten equal-width bands:

Measure	Value
Brier score	0.1362
Expected calibration error (ECE)	0.0301
Maximum calibration error (MCE)	0.1806
Mean predicted probability	0.2566
Actual base rate	0.2658
Global bias	-0.0093

The model is very slightly pessimistic overall, and an ECE of 0.03 is good. This is what licenses the retention simulator to treat its outputs as probabilities rather than as mere rankings.

The module has five documented skip paths — regression targets, estimators without `predict_proba`, multiclass problems, probability matrices with an unexpected column count, and single-class splits — and each records a stated reason rather than manufacturing a number.

One correctness note from the code deserves repeating, because the obvious implementation is wrong. Identifying the positive-class probability column as "the larger of the two labels" fails for `classes_ == [1, 2]`, where the probability column is `P(1)`; pairing it with `truth == 2` inverts the entire reliability curve and lands ECE near 1.0 *in silence*.

13.5 The model card

The card assembles everything above into eight sections and is governed by one rule:

A section with no evidence says so, in the card, with the reason. It is never silently dropped.

The reasoning is sharp: a card that omits its fairness section looks like a card whose fairness was fine — precisely the failure the artifact exists to prevent.

Two refusals are built in.

It never authors an intended-use statement. Intended use is a claim about which decisions a model is permitted to inform, and only a person who knows the domain can make it. The card states that it is absent and who must write it, rather than generating plausible prose about a use case nobody approved.

It refuses to pair one model's identity with another model's metrics. If the registry champion differs from the champion named in the evaluation report, test metrics and the selection narrative are withheld. Otherwise the card would print model X's name, version and data hash above model Y's test metrics — the most consequential possible failure in a document whose entire rule is "no evidence, say so".

14. Testing strategy

The suite contains **136 tests across 11 files**. Its guiding principle, from the trust test module's docstring:

The cases chosen are the ones that would ship a wrong number silently rather than crash.

File	Focus
<code>test_trust.py</code>	The trust layer — 64 tests, the largest by a wide margin
<code>test_metric_selection.py</code>	Degenerate metric values must never win a selection
<code>test_model_pipeline.py</code>	The governed modelling chain, in a redirected temporary workspace
<code>test_orchestrator.py</code>	Dependency refusal, status persistence, resume and force
<code>test_classification.py</code>	Task-aware metrics, multiclass handling, simulator identities
<code>test_schema.py</code>	Schema integrity and the EDA refusal guard
<code>test_automl.py</code>	Target-guessing and identifier heuristics
<code>test_splitter.py</code>	Stratified splitting and determinism
<code>test_features.py</code>	Feature proposal and deterministic execution
<code>test_imputer.py</code>	Train-only median imputation

Representative cases that encode real defects:

- `test_evaluation_touches_test_only_for_champion` — the untouched-test doctrine, enforced.
- `test_median_is_fit_on_train_only` — forces a NaN into validation and asserts it is filled with the *train* median.
- `test_governance_refuses_unapproved_model` — the approval gate actually raises.
- `test_nan_first/_middle/_last_does_not_win_champion` — three separate tests, because the bug was ordering-dependent.
- `test_group_without_positives_reports_tpr_as_null_not_zero` — undefined is not zero.
- `test_small_group_is_reported_but_excluded_from_gaps` — a four-row group is noise.
- `test_opposing_gaps_do_not_cancel`.
- `test_card_never_authors_an_intended_use_statement`.
- `test_card_refuses_to_pair_one_model_with_another_models_metrics`.
- `test_card_reports_no_subgroup_column_as_a_finding_not_a_pass`.
- `test_id_heuristic_does_not_eat_features_ending_in_id`.

A shared fixture generates a 400-row schema-conformant synthetic housing frame with a fixed seed, deliberately spreading `median_income` across all five stratification bins so every bin is

populated, and injecting eight missing values. The suite therefore runs with no network access and no downloaded data.

15. Application layers

15.1 The Streamlit analyst app

Nine pages covering the pipeline stages, including the retention simulator with its four campaign levers and live-updating outcome metrics.

15.2 The Next.js review console

A local-only Next.js 15 application over the same artifacts. It is explicitly not deployable to a host without a local Python environment — it is a local tool, not a hosted product.

The user journey is: drop a CSV on the front page, review the auto-detected target and task (editable, with the reasoning shown), run the analysis, then walk a ten-stage journey rail reading each stage's outputs, with downloads for the HTML report, PDF, Markdown briefing and CSVs.

Three engineering details are worth recording.

Stage-run detection is subtler than it looks. The first implementation checked whether a stage's output directory existed. But stages create their directories before doing work, so a crashed EDA stage left an empty directory and the application cheerfully reported "10 of 10 stages produced artifacts". The check now requires a *file* beneath the probe path.

The artifact endpoint has two independent gates: path containment, so a request cannot escape its workspace, and an extension allow-list — specifically to keep `.joblib` model files off HTTP.

The Python bridge emits JSON on stdout and nothing else. Because pipeline code logs freely, the bridge swaps `sys.stdout` for a throwaway buffer while the pipeline runs, and sends all logging to stderr. A failed run rolls back by discarding the partially created workspace.

16. Engineering lessons

The repository carries a checklist of traps found the hard way. They generalise well beyond this codebase.

1. NaN wins a ``max()`` or ``min()`` selection. Every comparison against NaN is `False`, so a naive selection loop silently keeps whichever candidate came first.

- 2. The task must be re-derived when the target changes.** Task is a property of the target column, not of the file. Changing the target requires a re-scan.
- 3. Stratified splits need more rows per class than the obvious minimum.** A three-way split needs enough members of every class to populate all three partitions.
- 4. Degenerate columns reach the estimator.** Zero-variance and near-constant columns must be guarded explicitly.
- 5. `positive_class` silently binarises a multiclass target.** A binary encoding applied to a three-class problem collapses two classes without complaint.
- 6. Module-level path constants freeze the active project.** Reading an environment variable at import time means import order becomes load-bearing.
- 7. Two callers of one pipeline will drift.** The CLI and the web path each kept their own call list, and the lists diverged.
- 8. Generated prose is domain-bound.** Narrative written for house prices is confidently wrong about churn.

17. Limitations

Stated plainly, because a limitation found by the author is worth more than one found by a reviewer.

Accuracy is modest on housing. R^2 0.667 means a third of the variance in house values is unexplained. The linear family cannot represent the coastal geography the EDA stage identified; the model approximates a curved price surface with a plane. This was accepted knowingly as the cost of interpretability, but it is a real ceiling.

The housing model margin is thin. Ridge beat the unregularised baseline by about \$100 of RMSE, with Lasso tied at the baseline. Three models within \$100 of each other is a weak basis for declaring a winner, and the honest reading is that the choice among them barely matters.

No unsupervised branch exists. The AutoML detector labels every target as regression or classification. A dataset genuinely suited to clustering is therefore forced through the regression family and produces a meaningless model rather than a refusal — the `mall-customers` workspaces are exactly this failure.

Generated narrative prose is domain-bound. The catalogue's suitability and limitation text is written for California housing. An auto-registered project can inherit justification text it never earned. The dataset-agnostic driver module avoids this by generating no narrative, but the catalogue prose remains housing-specific.

Model families are limited. No gradient boosting, support vector machines, k-nearest neighbours or neural models. The single-tunable-parameter constraint that keeps the tuner simple would need revisiting to add them.

Selection optimism is argued, not measured. The claim that hold-out tuning did not over-select rests on the small validation-to-test movement. That is evidence, but nested cross-validation would quantify it directly.

The retention simulator exists only in the Streamlit app. The Next.js console does not expose it.

Fairness auditing is descriptive by design. The system never labels a column a protected attribute and never judges whether a gap is acceptable. This is a deliberate scope boundary, but it means the artifacts require a human reader to become findings.

18. Future work

In priority order:

32. **Template the governance prose** from the dataset descriptor, so model cards and proposals describe the project actually being run.

33. **Add an unsupervised branch** — k-means with silhouette-based selection — with its own selection metric and trust checks, so clustering data is routed correctly rather than mis-modelled.

34. **Add one boosted-tree family** to test whether the single-tunable-parameter design survives contact with a model that genuinely needs several.

35. **Nested cross-validation** to quantify selection optimism directly.

36. **Port the retention simulator** into the Next.js console.

37. **Group-aware permutation importance**, permuting all dummies of a categorical together so a categorical's importance is meaningful.

19. Appendix A — Command reference

```
# Run everything, end to end
python main.py all

# Resume from where the last run stopped
python main.py pipeline

# Run a specific range of stages
python main.py pipeline --from train --to trust
```

```

# Force a full re-run
python main.py pipeline --force

# Individual stages
python main.py ingest
python main.py validate
python main.py preprocess
python main.py split
python main.py understand
python main.py eda
python main.py features
python main.py propose
python main.py approve          # --features / --models
python main.py train
python main.py evaluate
python main.py explain
python main.py drivers
python main.py trust            # --split test|validation --groups COL...
python main.py report

# The churn project
WORKBENCH_PROJECT=churn python main.py churn-all

# An uploaded dataset
WORKBENCH_PROJECT=<slug> python main.py autorun

# Tests
pytest

# The retention simulator's own self-check
python src/simulation/retention.py

```

20. Appendix B — Artifact inventory

Artifact	What it proves
models/<name>/<version>/model.joblib	The exact fitted object, with its preprocessor
models/<name>/<version>/metadata.json	Parameters, tuning history, metrics, data hash, seed
models/model_registry.json	Which model is champion
artifacts/final_model_selection.json	Champion identity plus validation and test metrics
artifacts/evaluation/model_comparison.csv	The ranked comparison behind the choice
artifacts/experiments/experiments.jsonl	Immutable history of every run
governance/approvals/model_proposal.json	What was proposed, and why
governance/approvals/model_approval.json	What was approved
governance/approvals/feature_approval.json	Which predictors may enter the matrix
governance/audit/audit_log.jsonl	Append-only trail, ten event types
governance/lineage/lineage.json	Input to script to output graph, with file

	hashes
<code>governance/decisions.json</code>	The five standing methodological decisions
<code>artifacts/trust/subgroup_fairness.{json,csv}</code>	Per-group metrics and gaps
<code>artifacts/trust/permutation_importance.{json,csv}</code>	Measured dependence, with noise flags
<code>artifacts/trust/calibration.json</code>	Brier, ECE, MCE, reliability table
<code>artifacts/trust/model_card.{md,json}</code>	The assembled account, including its own gaps
<code>artifacts/reports/project_report.html</code>	Self-contained report with embedded figures
<code>artifacts/workflow_status.json</code>	Per-stage status, duration and outputs

21. Appendix C — Repository map

<code>ai-data-science-workbench/</code>	
<code>main.py</code>	CLI entry point (19 subcommands)
<code>config/</code>	
<code>constants.py</code>	Seed, split fractions, thresholds
<code>paths.py</code>	Every path; resolves the active workspace
<code>active.py</code>	Task, target, metric for the active project
<code>datasets.yaml</code>	Built-in project descriptors
<code>src/</code>	
<code>data_manager/</code>	Ingestion, dataset registry, validation
<code>domain/</code>	Semantic schema, dataset value object, profiler
<code>preprocessing/</code>	Cleaning, splitting, imputer, encoder, scaler
<code>eda/</code>	Understanding, exploration, analysis summary
<code>feature_engineering/</code>	Proposal, transformers, execution
<code>model_proposal/</code>	Model catalogue and proposal document
<code>model_training/</code>	Design matrix, factory, metrics, trainer
<code>model_evaluation/</code>	Evaluator and comparator
<code>explainability/</code>	Interpretation, drivers, optional SHAP
<code>governance/</code>	Approvals, audit log, lineage, decisions
<code>trust/</code>	Fairness, permutation importance, calibration, card
<code>simulation/</code>	Retention simulator
<code>reporting/</code>	Report context and HTML renderer
<code>artifacts/</code>	Model registry, experiment tracker, artifact store
<code>automl/</code>	Detection, registration, auto EDA report
<code>workflow/</code>	Stage registry and orchestrator
<code>pipelines/</code>	Per-stage pipeline entry points
<code>tests/</code>	136 tests across 11 files
<code>app/</code>	Streamlit analyst application
<code>webapp/</code>	Next.js review console
<code>webapi/bridge.py</code>	JSON bridge between the console and Python
<code>docs/</code>	Methodology, plans, lessons, this report
<code>workspaces/<slug>/</code>	Per-project data, models, artifacts, governance

Closing statement

Every figure in this report can be traced to a file in the repository: which data produced it, which parameters, whose approval, and what the model gets wrong about which subgroup. On this project that traceability — not the accuracy — is the deliverable.

A model with an R^2 of 0.667 whose provenance is fully reconstructable is more useful to an organisation than one with an R^2 of 0.9 that nobody can reproduce, audit, or explain. The workbench is an argument for that position, made in code.